SkyRoute: Airline Ticket and Baggage Management System

DBMS Mini Project - Database Management System (CS/AI/AIM/DS202)

Database: skyroute_db | Tables: 18 | MySQL 8.0

Seed volume: 140 flight instances, ~14,000 bookings, ~17,000 baggage records (deterministic generator, seed 42).

Abstract

Managing flight bookings, seat allocation, gate assignment and baggage handling across several airports by hand causes duplicate seat bookings, gate clashes, wrong baggage-fee computation and inconsistent data between stations. SkyRoute is a centralized relational database for five Indian airports and three airlines that automates scheduling, reservations, baggage tracking, payments and check-in. The schema separates timetable templates (Flights) from dated departures (Flight_Instances); models Terminal/Gate/Runway infrastructure per airport; and organises staff with a supertype/subtype hierarchy. Integrity rules live inside the database: a unique seat-per-instance key, a capacity trigger, composite gate keys, and check constraints. A stored procedure books atomically, two views serve the departure board and the receipt (with overweight-baggage fee), transaction demos prove atomicity and isolation, and indexes with before/after EXPLAIN checks prove performance at volume.

1. Title Selection

SkyRoute: Airline Ticket and Baggage Management System - the real-world problem solved (ticketing and baggage across multiple airports), not a single-route shuttle.

2. Problem Statement

Manual single-station workflows suffer five failures, each closed by one named database mechanism:

- Duplicate seat booking -> UNIQUE(instance_id, seat_number).
- Gate clash -> UNIQUE(terminal_id, gate_number) plus gate rotation.
- Wrong baggage fee -> fee rule encoded once in booking_receipt_view.
- No single database -> one skyroute_db for all five airports.
- Overselling -> trg_booking_capacity trigger against aircraft capacity.

3. Objectives

- Automate scheduling and gate allocation across five airports.
- Make double booking and overselling impossible at engine level.
- Compute baggage fees deterministically from weighed kilograms.
- Answer boards, manifests, revenue and utilisation in single queries.
- Guarantee integrity with least-privilege access control.
- Prove performance at volume with indexing and cost analysis.

4. Requirements Gathering

Functional

- Add/edit/view airports, airlines, aircraft, flights, dated instances.

- Assign aircraft, gate, runway and crew per departure.
- Reserve seats, log baggage, record payments and check-ins.
- Manage staff and roles.

Non-functional

- Fast response at tens of thousands of rows.
- Integrity under concurrent sellers.
- Secure access.

5. Entity-Relationship Modeling

Entities: Airport, Airline, Aircraft, Flight, Flight_Instance, Terminal, Gate, Runway, Passenger, Booking, Baggage (weak), Payment, Check_In, Employee (supertype) with Pilot, Crew_Member, Ground_Staff subtypes, and Crew_Assignment linking staff to dated flights. Flight 1-N Instances; Airport 1-N Terminals 1-N Gates; Passenger 1-N Bookings N-1 Instances; Booking 1-N Baggage (identifying), 1-1 Payment/Check_In. See ER_Diagram.png.

6. Schema Design (Relational Model)

18 InnoDB tables. Surrogate integer PKs keep FKs narrow; business keys (IATA codes, tail numbers, licences, emails, txn refs) are UNIQUE. 1NF: masters remove repeating groups. 2NF: attributes depend on whole keys. 3NF: no transitive dependencies (city lives once in Airports). BCNF-clean: every determinant is a key.

Table	**PK**	**Key rules**
Airports	airport_id	UNIQUE iata, name
Airlines	airline_id	UNIQUE name, iata
Aircraft	aircraft_id	UNIQUE tail; CHECK cap 1..850
Flights	flight_id	UNIQUE number; 2 airport FKs; dep<>arr
Flight_Instances	instance_id	UNIQUE(flight, dep_time)
Terminals/Gates/Runways	own ids	UNIQUE per parent (anti-clash)
Passengers	passenger_id	UNIQUE passport, email
Bookings	booking_id	UNIQUE(instance, seat)
Baggage	(booking, seq)	weak entity; 0 < kg <= 100
Payments/Check_Ins	own ids	UNIQUE booking_id (1-to-1)
Employees+subtypes	employee_id	subtype PK = supertype PK
Crew_Assignments	assignment_id	UNIQUE(employee, instance)

7. SQL Implementation

DDL

01-travel-core.sql: 13 travel tables with keys, CASCADE where children cannot outlive parents, RESTRICT on airports (deleting an airport must never wipe flights). 02-staff-roles.sql: staff hierarchy plus DCL - sky_admin (all), sky_staff (read-write), sky_viewer (read-only), verified with SHOW GRANTS on the exact database name.

DML

generate.py (seed 42) writes seed_data.sql: 5 airports, 3 airlines, 10 aircraft, 10 flights, 140 instances over 14 days, 500 passengers, ~14k bookings at 35-80% load, ~14k payments, ~17k bags (20% overweight tail), ~5k

check-ins, 8 staff with assignments. Verified: zero duplicate seats, all FKs in range.

DCL

Least-privilege roles as above; viewer INSERT is denied (test R-b).

8. Advanced SQL Features

Views: flight_board_view (whole departure board, one query); booking_receipt_view (price + fee: 20 kg free, Rs 500/excess kg). Procedure MakeBooking inserts booking + Pending payment in one transaction with an exit handler (rollback + re-signal). Trigger trg_booking_capacity refuses inserts past aircraft capacity with SQLSTATE 45000. Join pack (10 queries) + analytics pack (GROUP BY, HAVING, CASE, subquery) cover every module.

9. Transaction Management

Demo A commits booking+payment together. Demo B rolls back on payment failure and the seat stays free (atomicity). Demo C uses two sessions to show REPEATABLE READ isolation. Script: 04-transactions.sql.

10. Indexing and Optimization

FK columns are already auto-indexed by InnoDB and were not re-indexed. Four workload indexes: booking_status, scheduled_departure, (last_name, first_name), weight_kg - each with a before/after EXPLAIN pair in the test log showing fewer rows examined.

11. Testing and Validation

TESTING.md: 30 checks in 8 groups - joins, analytics, constraint rejections (duplicate seat ERROR 1062, impossible route, bad weight, RESTRICT, CASCADE), procedure atomicity, transaction demos, EXPLAIN pairs, grants, scale counts. Each states its expected result.

12. Documentation

This report (docx + pdf), README.md (run order, users, rules, screenshot workflow), TESTING.md, decision journal C1-C13 in the SQL comments, ER diagram, slide deck, Flask brief report.

13. Presentation and Reporting

This report, 11-slide deck, all sources, README, test log, 2-3 minute demo video (departure board, live booking, duplicate-seat refusal, fee receipt).

Conclusion

Every problem item is closed by a named mechanism, every mechanism has a test, and the system holds production-shaped volume while answering analytics in single queries. The Flask project reuses this design on SQLite with raw SQL and full CRUD.

References

- Ramakrishna R., Gehrke J. Database Management Systems, 3rd Ed.
- Silberschatz, Korth, Sudarshan. Database System Concepts, 7th Ed.
- MySQL 8.0 Reference Manual: constraints, triggers, EXPLAIN.